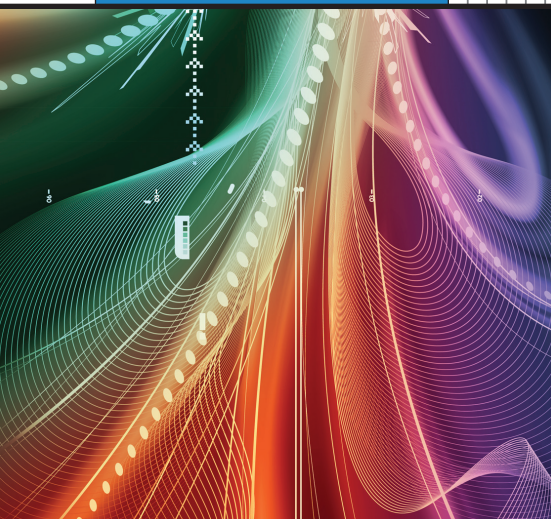




Rethinking Hardware-Software Codesign for Exascale Systems

John Shalf, *Lawrence Berkeley National Laboratory*

Dan Quinlan, *Lawrence Livermore National Laboratory*

Curtis Janssen, *Sandia National Laboratories*

The US Department of Energy's exascale computing initiative has identified hardware-software codesign as a central strategy in achieving more agile hardware development. Hardware simulation and code analysis tools that facilitate deeper collaboration between hardware architects and application teams will be an essential component of the codesign process.

Over the years, users of high-performance computing (HPC) systems have seen dramatic increases in peak performance claims but with no commensurate improvement in application performance. Moreover, as these systems approach the exaflops scale, their designers are confronting enormous electrical power requirements. The cost of power is expected to exceed the procurement costs of HPC systems, and will thus ultimately limit their future practicality.

Unless system designers work aggressively with vendors and the scientific community to develop more energy-efficient solutions, these trends will lead to an HPC industry crisis.^{1,2} Traditional HPC system design methodologies have not had to account for power constraints or parallelism on the level designers must contemplate for exascale systems. Furthermore, these tectonic shifts in computer architecture are anticipated to radically change the programming model and software environment at all scales in future computing systems. The designers of HPC hardware and software components have an urgent need for a sys-

tematic methodology that reflects future design concerns and constraints.

The US Department of Energy's (DoE's) exascale computing initiative has identified hardware-software codesign as a central strategy to meet this need. The idea is to have a novel development partnership in which application scientists participate in a highly collaborative and iterative design process well before the vendor brings the system to market. Accelerating this cycle could improve energy efficiency and usable performance by orders of magnitude.

Through a joint project involving our three laboratories, we are assembling Codesign for Exascale (CoDEX), a comprehensive hardware-software codesign environment that will enable an unprecedented opportunity for application and algorithm developers to influence the direction of future architectures so that they meet DoE mission needs (<https://sites.google.com/a/lbl.gov/codex>). CoDEX combines three elements:

- highly configurable, cycle-accurate simulation of node architectures from Lawrence Berkeley National Laboratory's Green Flash project (www.lbl.gov/cs/html/greenflash.html);
- novel automatic extraction and exascale extrapolation of memory and interconnect traces using the Lawrence Livermore National Laboratory's ROSE compiler framework (www.rosecompiler.org); and
- scalable simulation of massive interconnection networks using Sandia Laboratories' Structural Simulation Toolkit (SST)/macro, a coarse-grained simulator (<http://sst.sandia.gov>).

These tools will enable a tightly coupled software-hardware codesign process that is highly suitable for complex HPC designs.

WHY CODESIGN AND SIMULATION TOOLS ARE KEY

HPC system development requires a more tightly integrated approach to designing future computing systems to meet the extreme power and performance challenges for the next decade. Codesign is an iterative process, in which the goal is to optimize both applications (algorithms and algorithmic implementations) and hardware for some combination of performance, power, and cost. An essential first step is to identify the application targets, which makes it possible to use target applications' performance on the target machine as a key success criterion—one in which hardware architects and applications teams are equally invested.

An effective codesign process also requires accurately predicting the performance, cost, and power consequence of any design tradeoff in algorithms or hardware configuration. The architectural simulation of computing systems plays an essential role in providing this predictive capability because it enables the quantitative exploration of HPC system design tradeoffs. HPC system development requires a more tightly integrated approach to designing future computing systems to meet the extreme power and performance challenges for the next decade.

The embedded processor market has refined codesign processes over the past 20 years to meet the demanding cost and power efficiency requirements of battery-powered consumer electronics (smartphones, MP3 players, and so on) as well as of power-sensitive high-performance embedded applications like avionics systems. Codesign's success is due largely to a continued focus on developing tools that make the methodology productive, cost-effective, and beneficial, such as automated processor synthesis tools, cycle-accurate simulators, and the automated generation of software tools (compilers and debuggers) from hardware specifications. These tools have broken through the slow pace of system design that held back the embedded industry for many years and that continues to impede critical advances in HPC design and implementation.

The partnership in DoE's vision furthers codesign practice by anticipating application needs. The multidisciplinary codesign team begins the process by identifying leading-edge, high-impact scientific applications and then using these as optimization targets instead of the traditional speed and feed targets (flops and bandwidth) and percentage of peak flops. The design process is application driven. Thus, rather than looking at what scientific applications can run on an exascale system after it ar-

rives, the team seeks to uncover what a system needs to meet the requirements of critical science problems. There was no thunderclap or equivalent to a sonic boom when we crossed the threshold to petaflops, nor will there be for the first system to exceed the "exaflops barrier." The success of the DoE exascale program hinges on the success of the science applications. As such, the process builds on DoE's understanding of application requirements and expands its already broad-based computational science portfolio.

In contrast to a traditional pipelined process, which optimizes isolated team-specific metrics, the codesign process requires collaboration and compromise from all teams to achieve a common optimization target. Application experts work in concert with computer architects and algorithm designers—all using sophisticated tools to accelerate the design loop to meet the common goal of maximizing scientific application performance



Codesign is an iterative process, in which the goal is to optimize both applications and hardware for some combination of performance, power, and cost.

within the constraints of power and cost. The methodology depends on a bidirectional optimization of design parameters, in which software requirements drive hardware design decisions, and hardware design constraints motivate changes in the software design to better fit within those constraints.

Thus, a deep analysis of application requirements drives key design decisions for the overall system architecture. It provides quantitative measures of application requirements and relates them to architectural parameters such as on-chip memory, memory bandwidth demands, and interconnection requirements.

Unfortunately, many hardware design choices are too costly to implement or do not improve energy efficiency enough to justify the cost—a condition that motivates software implementation changes. Architectural simulation provides an avenue to experiment with hardware configurations, programming models, and algorithms on exascale class machines before making the full system implementation available. Designers can integrate models of the hardware, runtime system, and application into the simulation to evaluate the tradeoffs among the design choices imposed on each aspect of the integrated system. Cycle-accurate simulation tools let designers quantify the performance impact on applications in light of specific hardware constraints.

Table 1. Application surrogates and their definitions.

Surrogate	Description
Compact application	Small app with fewer features and simplified boundary conditions relative to a full app
Mini-application	Small, self-contained program that embodies essential performance characteristics of key apps
Skeleton application	Program that captures an app's control flow and communication pattern; can be run only in a simulator
Proxy application	General term for all other surrogates
Mini-driver	Small programs that act as drivers of performance-impacting library packages
Kernel	Program that captures an algorithm's node-level aspects

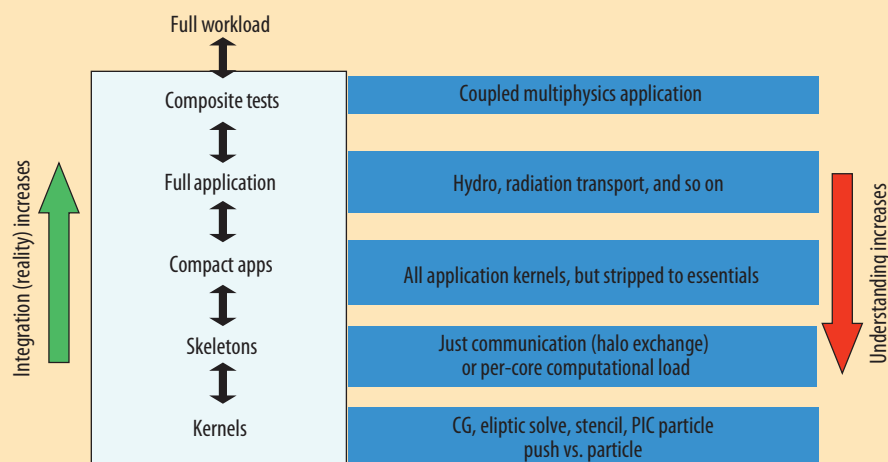


Figure 1. Interplay among the layers in the application hierarchy. In the CoDEx co-design process, kernels enable the rapid exploration of new languages and algorithms because of ease of rewriting, whereas full applications are harder to rewrite but ensure adherence to original application requirements.

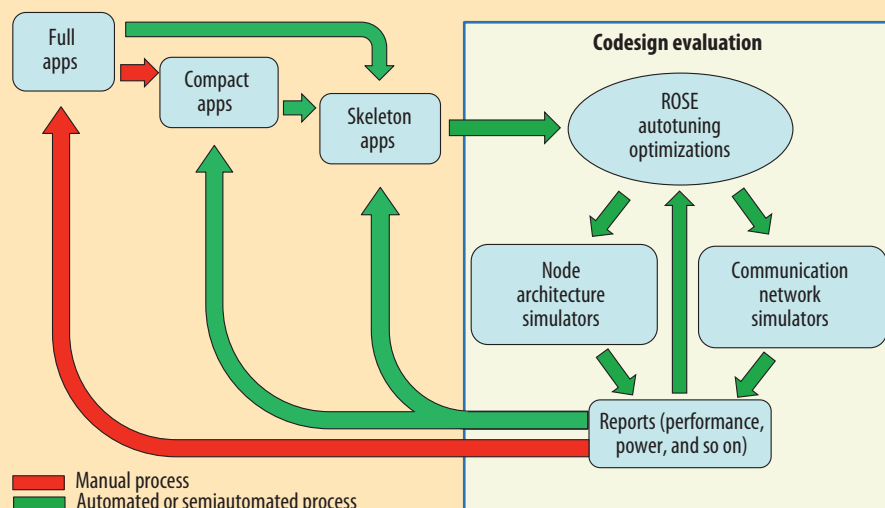


Figure 2. Hierarchy of code representations. The CoDEx codesign process uses a hierarchy of simplified surrogate code representations to provide hardware designers with actionable detailed information while still ensuring that the context for any insight remains faithful to the full application's requirements.

Analysis of the hardware constraints in terms of cost, area, and power consumption serves as feedback to motivate changes in the application and algorithm design to better fit within hardware constraints.

CODESIGN TOOLS

The codesign process requires both hardware and software models. The hardware models capture the structure and function of the exascale system's hardware components. The software models consist of a variety of surrogates for the application software. Different surrogate types allow designers to implement different aspects of the codesign process. Table 1 lists several surrogates and their definitions. Figure 1 shows how the application reduction hierarchy relates to code analysis—as applications reduce to the kernel surrogate, understanding increases, and as the design moves up the hierarchy, its gets closer to reality, or the full application.

Figure 2 shows the codesign workflow using these different application representations. Compact mini-applications and mini-drivers and kernels run on actual hardware; skeleton applications run only in a simulated hardware environment. Tools that allow automated conversion among these surrogates are a key feature of the CoDEx environment.

Deep code analysis

In our codesign process, the ROSE infrastructure forms a central part of the tool chain for automating the extraction of skeleton applications to support input to the simulators. As Figure 3 shows, the chain starts with the ROSE-based tool reading source code into an abstract syntax tree using an intermediate representation. The tool then analyzes the source code indirectly using the AST and subsequently transforms the AST to cut away parts of the application's representation to reduce complexity but preserve specific features, such as message-passing interface (MPI) communication. From the modified AST, the tool can generate a less complex source code version, which becomes the skeleton for input into hardware simulators.

Research to provide a deep understanding of code performance depends on significant internal compiler analysis. Current work uses data dependence and control-flow information (def-use analysis) to support the generation of backward slices rooted in the application's MPI calls. Future work will expand on def-use analysis and use a more general dataflow framework in ROSE to support the generation of skeletons that capture a range of behaviors—not just MPI communication. Future work will also incorporate interprocedural analysis based on the system dependence graph to track the use of variables across function interfaces and provide more aggressive simplification levels for large-scale applications.

These techniques to automate generation of skeleton applications are central to how codesign can incorporate current scientific software into the process of evaluating modern numerical techniques for future exascale architectures. Research to support this work is an ongoing collaborative effort of the ROSE project and Galois Inc., with iterative results available on the ROSE project website (www.rosecompiler.org).

Coarse-grained interconnection simulation

Coarse-grained simulations let designers study large-scale systems in a way that captures the complex interactions among hardware components, including interactions that arise only at the largest scales. The SST/macro³ and Warwick Performance Prediction simulator⁴ are examples of simulators designed specifically for studying exascale class machines. Our codesign process uses

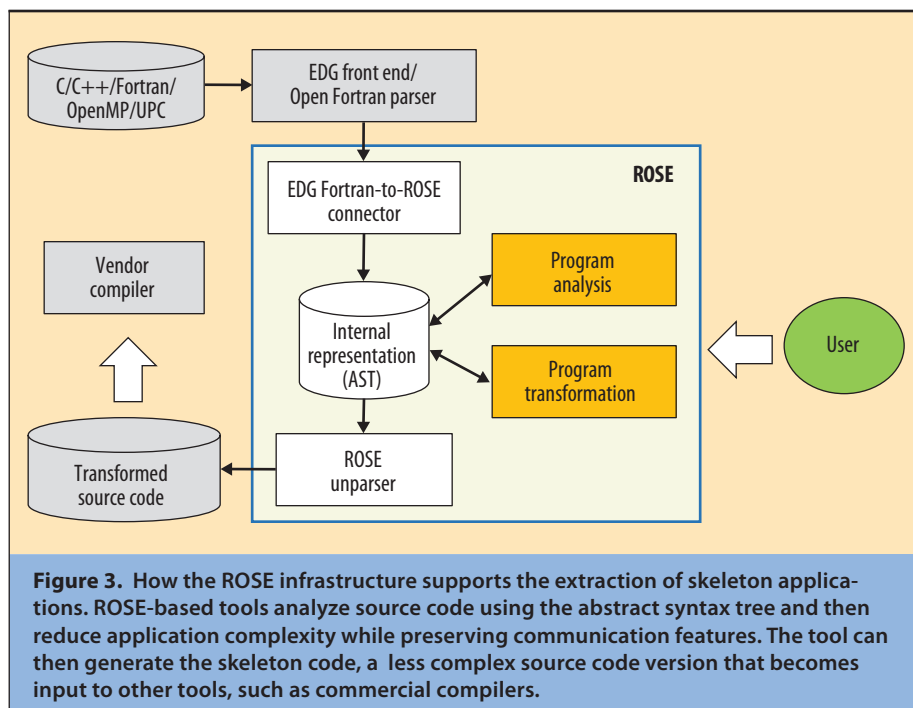


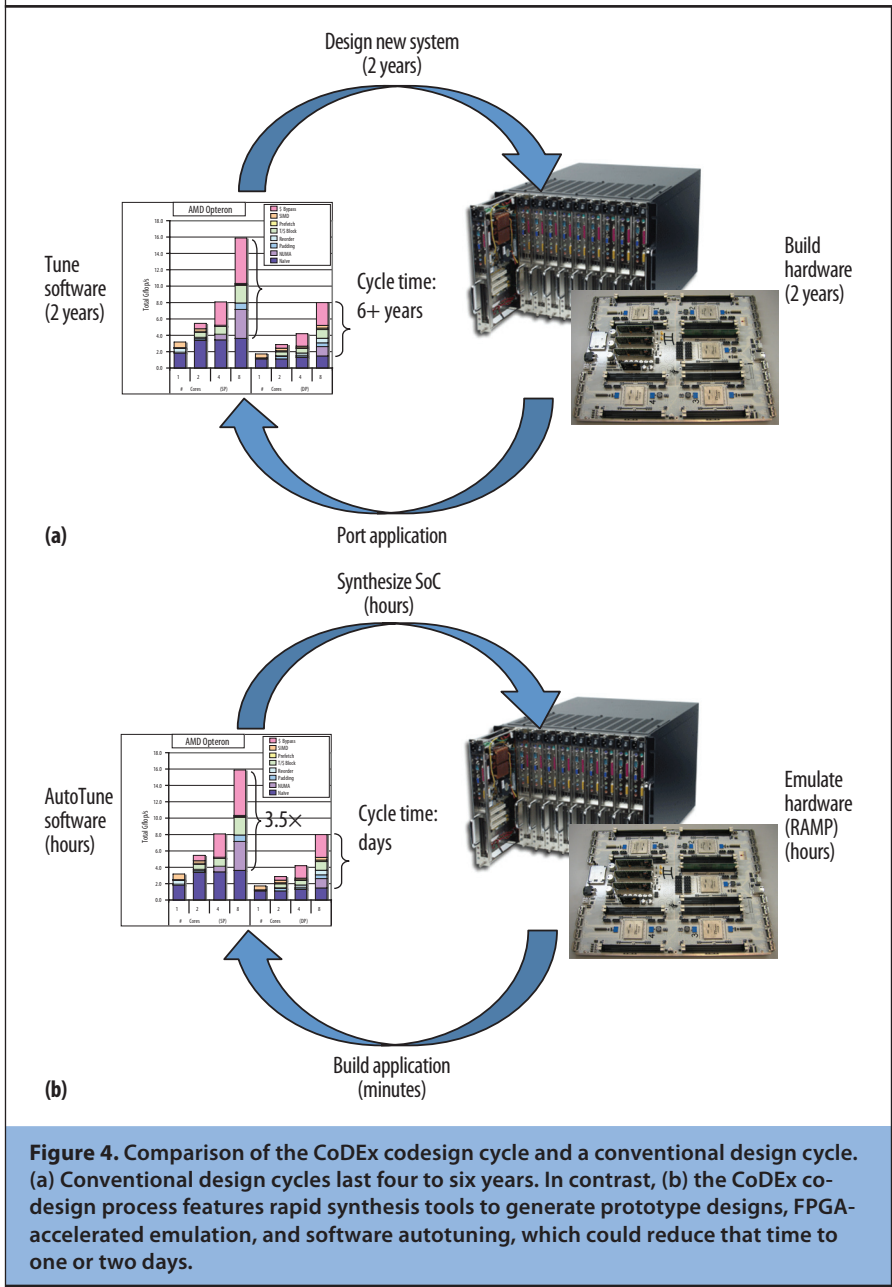
Figure 3. How the ROSE infrastructure supports the extraction of skeleton applications. ROSE-based tools analyze source code using the abstract syntax tree and then reduce application complexity while preserving communication features. The tool can then generate the skeleton code, a less complex source code version that becomes input to other tools, such as commercial compilers.

the SST/macro simulator. The simulator software is modular, permitting the study of various computational and communication models. SST/macro is distributed under an open source license and is downloadable from <http://sst.sandia.gov>.

Designers can use the simulator in two ways. One is to replay a trace of a previously run MPI application through the simulator, allowing its execution time to be estimated on new hardware. The application mimics a real application's control-flow and messaging pattern except with values from performance models instead of the computational and message-passing costs.

Alternatively, the simulator can use a skeleton application^{5,6} that domain experts provide or ROSE-based analysis tools generate. Traditionally, an application analyst must manually extract skeleton applications from full applications by removing all noncommunication code yet preserving all the code necessary to compute the communication topology and message sizes. This task is tedious and labor intensive. Because communication skeletons are central to modeling communication performance, the CoDEx project team has developed technology that enables the ROSE compiler to automate a skeleton's extraction.

Both ways of using the simulator have unique benefits. The tracing approach provides an easy way to understand existing application performance on new architectures. The skeleton application approach allows studies at extreme scales and permits the evaluation of new programming model ideas and notional algorithms without a complete implementation.



Rapid design synthesis

The embedded processor market has refined codesign processes by developing tools that make hardware-software codesign productive, cost-effective, and beneficial. Our codesign approach extends many of the tools designed for the rapid synthesis of embedded designs so that they are suitable for exploring HPC design alternatives.

One of our key extensions is to the Tensilica Xtensa Processor Generator (XPG) tool chain to work as our rapid prototyping platform for the HPC simulation. XPG provides an end-to-end solution for quickly creating simple, semi-custom processors from optimized, power-efficient building blocks. The XPG’s customizable instruction set,

communication interfaces, and memory hierarchy make it ideal for exploring novel chip multiprocessor designs. Its ability to extend the instruction set to add application-specific functionality produces a streamlined processor with scratchpad memories, advanced communication features, and custom operational codes that facilitate advanced communication and synchronization. XPG's ability to automatically generate C/C++ compilers, debuggers, and functional models enables fast software porting and rapid testing with a new architecture. It also speeds language development by enabling researchers to target source-to-source translation technology (targeting the compilers that XPG automatically generates), which is substantially easier than crafting a new compiler from scratch.

Accelerated hardware emulation

Design synthesis tools such as XPG produce gate-level Register Transfer Language, a list of gates with the circuit connections between them. Designers can use gate-level RTL as a chip-design layout, which in turn can be mass-produced at a chip fabrication facility. An alternative is to load the gate-level RTL for a potential processor design onto field-programmable gate arrays (FPGAs) for full cycle-accurate emulation. Architectural emulation platforms based on FPGAs, such as Palladium,

are a common way to support the rapid prototyping and evaluation of application-specific IC designs. Once the design synthesis tool maps gateway components onto the FPGA's programmable logic blocks, the resulting system looks for all practical purposes like the actual hardware, except that it runs at a much lower clock frequency (but orders of magnitude faster than software simulation). The emulated system can boot conventional operating systems such as Linux and use the actual compilers targeted for the final development platform.

FPGA emulation is hundreds to thousands of times faster than typical software emulation environments, yet its dramatic speed increase does not sacrifice accuracy. On

the contrary, FPGA emulation is arguably more accurate than a software simulation environment, since it truly represents the hardware design. The direct mapping to FPGAs on the hardware emulation platform and copious performance data provide a fast, accurate performance emulation environment. In this environment, designers can benchmark real codes, thus ensuring that application developers are intimately involved in the codesign process.

Gate-level RTL design also provides accurate feedback on the design's power consumption because it reflects the actual gates that would produce the real chip design. In contrast, power estimation is much more difficult to verify in a software-only simulation environment.

Our codesign process adopts the Research Accelerator for Multiple Processors (RAMP) technology, the result of a cooperative effort among six universities to build a novel emulation system for parallel processors (<http://ramp.eecs.berkeley.edu>). RAMP emulation technology uses the Berkeley Emulation Engine 3 (BEE3) boards, which in turn use Xilinx FPGAs. Each board has inter-FPGA links on the same board and between boards. Consequently, designers can scale up to simulate a single many-core socket, or even larger-scale clusters. The Berkeley RAMP Blue project demonstrated emulation of more than 1,000 cores using a stack of 16 BEE2 boards.⁷

With FPGA-accelerated emulation capabilities, a designer can generate a new system design (tape out) every day; system build time is in minutes instead of years. Designers can also evaluate alternative design scenarios in minutes—which used to take months or even years with conventional development processes.

Overall, the performance and increased confidence in the timing model of FPGA-accelerated hardware emulation platforms enhance interaction at all levels of system development. As Figure 4 shows, these capabilities can facilitate and ultimately accelerate the iterative process of science-driven system design.

CODESIGN STRATEGY

Codesign depends on a strategy that involves integrated design teams as well as sophisticated tools. Application experts work in concert with computer architects and algorithm designers to accelerate the design loop. The optimization target for the process is the delivered application performance for the combined hardware and software environment, rather than tangentially related metrics such as peak flops or byte-to-flops ratios. Therefore, it is essential to clearly identify the application up front that will serve as the common measure of success for all aspects of the system design.

The iterative design methodology in Figure 4 depends on a bidirectional optimization of design parameters: deep analysis of application requirements drives key design decisions for the overall system architecture, while ar-

MULTIRESOLUTION MODELING

Modeling, simulation, and compiler analysis all play synergistic roles in the codesign process to cover a broad space of design parameters. As Figure A shows, a multiresolution approach using multiple modeling methodologies is essential to cover the extreme scale and provide enough fidelity to inspire confidence in the design choice.

Tools such as cycle-accurate hardware simulation offer extreme detail in their modeling capability, but limit system scale to node size or small clusters. Software simulators, such as SST/macro, can expand the system scale that a model can handle, but they must neglect some detail to achieve that scalability. Likewise, constitutive models and other empirical modeling methods can cover much larger systems, but by definition can model the effects only as parameters. Most modeling is through empirical models because they are faster to construct and evaluate, but software-based and cycle-accurate models are also necessary because they can verify that the simpler model has included all the important effects and has not neglected anything essential but unanticipated.

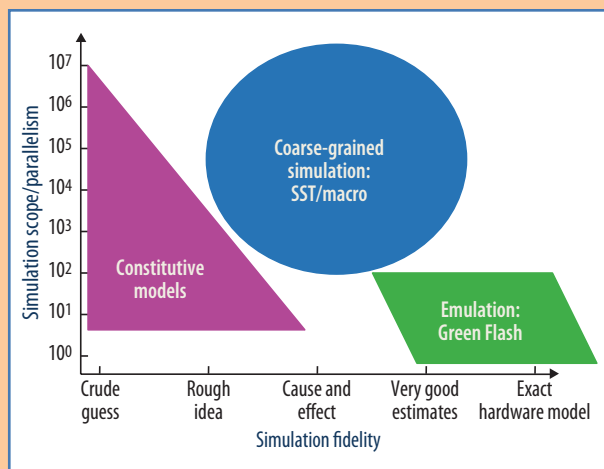


Figure A. Multiple modeling approaches cover both the scale and accuracy required to understand system design tradeoffs. The two key axes for simulation and modeling techniques are model fidelity (horizontal axis) and the scale of the system that simulation can handle.

chitectural simulation provides quantitative feedback on potential design tradeoffs for both hardware and software implementations. This tightly integrated design optimization cycle will enable orders-of-magnitude improvement in energy efficiency and usable performance.

RESULTS

We have applied our codesign process in several projects to develop climate modeling and seismic imaging applications, as well as to automatically cotune hardware and software in the same process.⁸

Our first project was Green Flash, the codesign of a machine for kilometer-scale climate modeling.⁹ The three central features of the Green Flash design process were fast

Table 2. Comparison of energy efficiency in the Intel Nehalem, Nvidia Fermi, and Green Wave, a codesigned many-core processor implementation.

Architecture	Intel Nehalem	Nvidia Fermi	Green Wave
Total nodes	127,740	66,823	75,968
Megapoints per watt	4.27	6.28	32.63
Communication overhead (%)	9.50	43.00	16.00
Total megawatts	38.20	26.10	5.00

FPGA-based hardware emulation, rapid design synthesis tools, and software autotuning technology.

Green Flash benefited greatly from FPGA-based emulation to facilitate tightly coupled codesign processes. These processes in turn enabled the rapid development of an application-driven many-core chip design targeted at scientific applications. Finally, as the “Multiresolution Modeling” sidebar describes, the project demonstrated the effectiveness of using a multidisciplinary hardware-software codesign process that facilitates close interactions among application and computer scientists and hardware engineers in developing a system tailored to scientific computing requirements.

Another project, Green Wave, conducted in collaboration with the Fraunhofer Institute for Industrial and Applied Mathematics (www.itwm.fraunhofer.de), extended the codesign process to study reverse-time-migration problems in seismic imaging—among the most demanding commercial applications for large-scale HPC clusters.¹⁰ The team modeled the full node design, including the memory subsystem, as well as the interconnection design’s performance. Green Wave demonstrated how a hardware-software codesign process can achieve an order-of-magnitude increase in energy efficiency over comparable GPU- and CPU-based systems.

Table 2 shows how the codesigned Green Wave implementation compares with the Intel Nehalem and Nvidia Fermi designs for the seismic imaging survey. To ensure a fair comparison, the Green Wave design used a 45-nm process technology, the same memory technology as the Nehalem, and limited chip area to 240 mm², about the same as the die size for the Nehalem and about half the die size for the Fermi.

The project team optimized Green Wave using a co-design process for a typical seismic imaging survey. The survey, which must complete in a week, requires approximately 12,000 separate data analysis runs performed on a 30 × 20 × 10 megapoint computational grid. The first row of Table 2 shows the system size required to meet this computational rate; the last row indicates how many megawatts a system of that size would consume. The metric of value for these simulations is the number of megapoints per second—the number of grid points that the HPC cluster can process per second for this seismic survey. The second row of the table translates this into an energy efficiency metric of megapoints per watt of power consumed. The third row indicates how much of the computational time is spent on inter-processor communication using the same InfiniBand interconnection technology. (We collected detailed communication timing information from an instrumented MPI implementation.)

As the table indicates, relative to a conventional HPC design, the codesigned system offers 5 to 7.6 times higher performance given similar constraints on chip area, lithographic scale, and memory and interconnection technology. We believe expanding the codesign process to these components could yield another order-of-magnitude improvement, since the Green Wave processor performance optimizations shift the primary sources of energy consumption to the memory and interconnection components.

Our experience using architectural simulators reflects six years of prototyping application-tailored chip designs for the Green Flash (kilometer scale climate modeling) and Green Wave (high-resolution seismic imaging) projects. We have generalized the tools and methodologies developed to accelerate the codesign process for those projects, and these generalizations provide many of the hardware simulation capabilities for the CoDEx project.

Both the Green Flash and Green Wave codesign projects relied on rapid synthesis to prototype new interprocessor communication and synchronization services to support advanced execution models. For example, we synthesized direct word-granularity interprocessor communication services to support fine-grained synchronization primitives

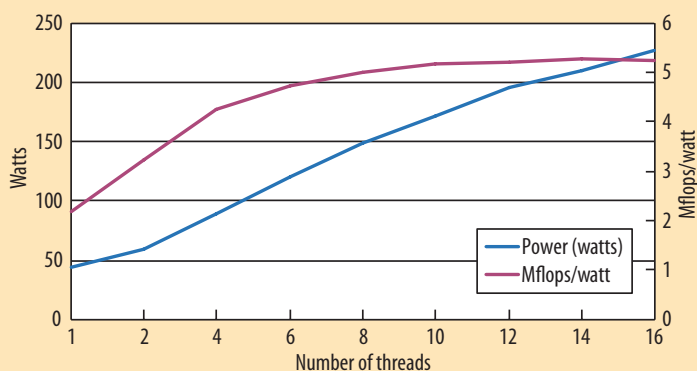


Figure 5. OpenMP transformations using ROSE. The evaluation used a simulator on a numerical kernel (Jacobi iteration), evaluating both power and Mflops per watt for various numbers of threads.

for noncache-coherent global-address-space languages. The more flexible and generalized CoDEx environment can automatically expose this new capability in the compiler to make it immediately accessible to our software stack developers for experimentation. These automated rapid prototyping tools will enable us to make substantial progress in developing novel hardware support for locality management, resilience, security, and interprocessor communication on a limited budget.

Advanced compiler environments also play a significant role in our architectural analysis. For example, we use ROSE to analyze sample applications and extract the MPI use as an input code to support the evaluation of message passing, which in turn supports the SST/macro simulator. This work supports the CoDEx project's charter to automate the application of codesign to support the evaluation of DoE applications.

The simulator represents a range of proposed future node architectures, with ROSE automating the complex code transformations needed to evaluate the potential of each design point. Figure 5 shows both power and Mflops per watt for different numbers of threads using a node simulator. Automating the exploration of both hardware and software design alternatives is essential in the goal to accelerate the iterative codesign cycle.

Codesign for exascale systems provides research challenges along all axes of the hardware, system software, and application design space. Our codesign process features hardware simulation and compiler-driven code analysis for model development, which allows designers to consider complete application codes rather than proxy kernels. Deep code analysis using compiler-assisted model development, interconnection simulation, and both software and FPGA-based hardware emulation systems with rapid design synthesis tools plays a central role. The cotuning environment enables the rapid search of many design alternatives to find the optimal hardware-software solution. Such an extensive search would not be practical or even possible with traditional decoupled design processes.

We are rapidly evolving the architectural simulation framework we have brought together in the CoDEx project to improve usability, increase integration with other simulators, model faults and power consumption, support alternative programming models, and quantify the uncertainty in our simulation results. These efforts will provide a platform for ROSE-based tools to analyze ever more complex application programs. Above all, we are placing a heavy emphasis on validation to ensure that our simulation results provide an accurate basis for exascale system design.

To simulate failure modes, we are expanding our ability to automatically inject faults into any system design com-

ponent. Designers can use this feature to harden software infrastructure in the face of transient errors or to test new models for fault detection, resilience, and recovery. Similarly, they can use SST/macro to insert simulated hardware failures into the interconnection model to test detection and correction methods for the interconnection protocol.

Our hardware simulation environment enables us to better understand how to build a programming environment together with the hardware that can exploit new features. For example, we are planning to study language and hardware design for invasive uncertainty quantification, in which probability density functions represent floating-point values. Another direction is to deepen the understanding of the programming involved in integrating NVRAM directly into the memory hierarchy—a likely trend for future exascale system designs. Hardware simulation enables us to explore the consequences and benefits of adding more intelligence to the memory chips to perform operations on data in situ within the memory subsystem.

In short, it is an exciting time to be involved in computer architecture, and the future of codesign for exascale computing is bright. We envision processes that unite multidisciplinary teams of application scientists, mathematicians, and computer architects in creating machines that can tackle problems critical to societies worldwide. 

Acknowledgments

The work described in this article was supported by the US Department of Energy's Office of Advanced Scientific Computing Research. Lawrence Berkeley National Laboratory is supported by the DoE Office of Advanced Scientific Computing Research under contract DE-AC02-05CH11231. Lawrence Livermore National Laboratory is supported by the DoE Office of Advanced Scientific Computing Research under contract DE-AC52-07NA27344. Sandia National Laboratories is a multiprogram laboratory managed and operated by Sandia Corporation, a wholly owned subsidiary of Lockheed Martin Corporation, for the DoE's National Nuclear Security Administration under contract DE-AC04-94AL85000. We also acknowledge Chris Rowen of Tensilica and Martin Deneroff for insightful guidance and support.

References

1. P. Kogge et al., "Exascale Computing Study: Technology Challenges in Achieving Exascale Systems," IPTO tech. report TR-2008-13, 2008, DARPA; rwww.cse.nd.edu/Reports/2008/TR-2008-13.pdf.
2. K. Asanovic et al., "The Landscape of Parallel Computing Research: A View from Berkeley," tech. report UCB/EECS-2006-183, 2006, EECS Dept., UC Berkeley; www.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-183.pdf.
3. C.L. Janssen et al., "A Simulator for Large-Scale Parallel Architectures," *Int'l J. Parallel and Distributed Systems*, vol. 1, no. 2, 2010, pp. 57-73.
4. S.D. Hammond et al., "WARPP: A Toolkit for Simulating High-Performance Parallel Scientific Codes," *Proc. 2nd Int'l Conf. Simulation Tools and Techniques (Simutools 09)*, Inst.

for Computer Sciences, Social-Informatics, and Telecommunications Eng., 2009, pp. 1-19.

5. V.S. Adve et al., "Compiler-Optimized Simulation of Large-Scale Applications on High-Performance Architectures," *J. Parallel and Distributed Computing*, vol. 62, no. 3, 2002, pp. 393-426.
6. R. Susukita et al., "Performance Prediction of Large-Scale Parallel Systems and Applications Using Macro-Level Simulation," *Proc. Int'l Conf. High-Performance Computing, Networking, Storage, and Analysis (SC 08)*, ACM, 2008, pp. 1-9.
7. A. Krasnov et al., "RAMP BLUE: A Message-Passing Many-core System in FPGAs," *Proc. Int'l Conf. Field Programmable Logic and Applications (FPL 07)*, 2007; <http://ramp.eecs.berkeley.edu/Publications/RAMP%20Blue%20FPL%202007.pdf>.
8. M. Mohiyuddin et al., "A Design Methodology for Domain-Optimized Power-Efficient Supercomputing," *Proc. Int'l Conf. High-Performance Computing, Networking, Storage, and Analysis (SC 09)*, ACM, 2009, pp. 1-12.
9. D. Donofrio et al., "Energy-Efficient Computing for Extreme-Scale Science," *Computer*, Nov. 2009, pp. 62-71, 2009.
10. J. Krueger et al., "Hardware-Software Codesign for Energy-Efficient Seismic Modeling," *Proc. Int'l Conf. High-Performance Computing, Networking, Storage, and Analysis (SC 11)*, ACM, to be published, 2011.

John Shalf is a staff computer scientist at Lawrence Berkeley National Laboratory and leads the Advanced Technology Group at the National Energy Research Supercomputing Center. His research interests include computer architecture, programming models, and frameworks for large-scale scientific application development. Shalf received an MS in electrical and computer engineering from Virginia Tech. He is a member of the IEEE Computer Society and the American Association for the Advancement of Science. Contact him at jshalf@lbl.gov.

Dan Quinlan is the leader of the ROSE project at Lawrence Livermore National Laboratory's Center for Advanced Scientific Computing. His research interests include program analysis, source-to-source compiler construction for general-purpose languages, programming models, parallel algorithms, and many-core and cache-based optimizations. Quinlan received a PhD in computational mathematics from the University of Colorado. Contact him at dquinlan@llnl.gov.

Curtis Janssen is a Distinguished Member of the Technical Staff at Sandia National Laboratories. His research interests include technologies required for high-performance scientific computation: programming models, performance analysis, and machine architecture. Janssen received a PhD in theoretical chemistry from the University of California, Berkeley. Contact him at cljanss@sandia.gov.

Enroll now.

ON THIS BATTLEFIELD, EDUCATION IS YOUR BEST DEFENSE.

Cyber attacks are being waged all over the world, creating an unprecedented demand for trained professionals to protect our country's data assets and develop cybersecurity policies. Help meet the demand with a bachelor's or master's degree in cybersecurity. Whether you plan to work for Cyber Command taking down cyber terrorists or for private industry battling hackers, UMUC can help you make it possible.

- Designated as a National Center of Academic Excellence in Information Assurance Education by the NSA and DHS
- BS and MS in cybersecurity and MS in cybersecurity policy available
- Programs offered entirely online
- Interest-free monthly payment plan available, plus financial aid for those who qualify

CYBERSECURITY

800-888-UMUC • umuc.edu/cyberwarrior



Copyright © 2011 University of Maryland University College